Maven, IntelliJ Build & Run Guide

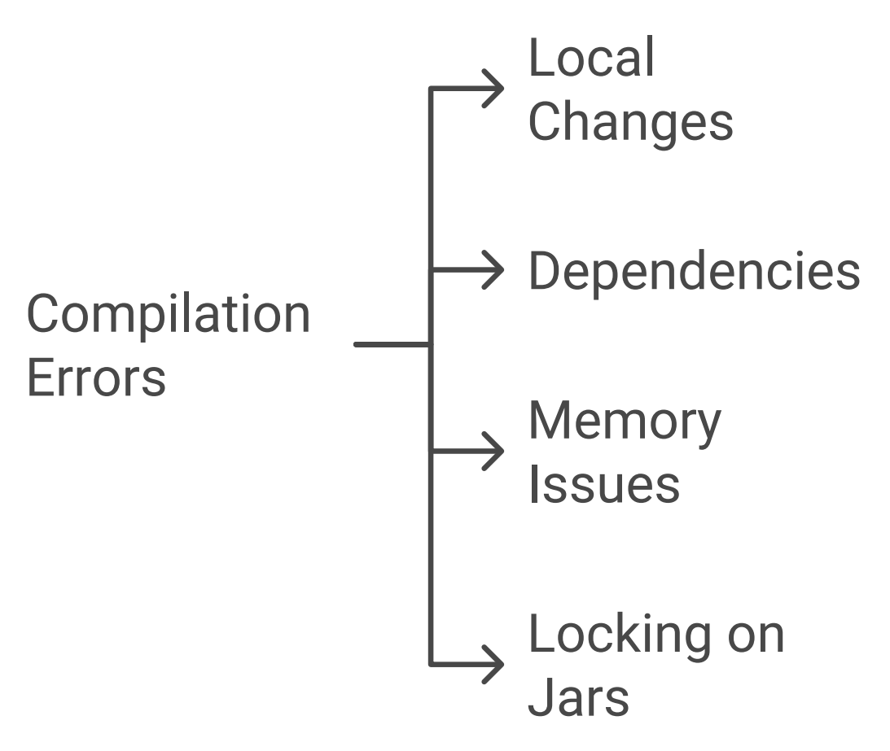
This document provides a comprehensive guide on how to effectively build and run projects using Maven and IntelliJ. It outlines the necessary steps, potential issues that may arise during the build process, and their corresponding solutions. By following these guidelines, developers can streamline their workflow and minimize errors during compilation and execution.

1. Starting with Maven

Always start with either **MVI** (**mvn install**) or **MVC** (**mvn clean install**).

Potential Problems

1. **Compilation error due to local changes**
2. **Compilation error due to dependencies**
3. **Compilation problems due to memory**
4. **Compilation problems due to locking on jars**



Solutions

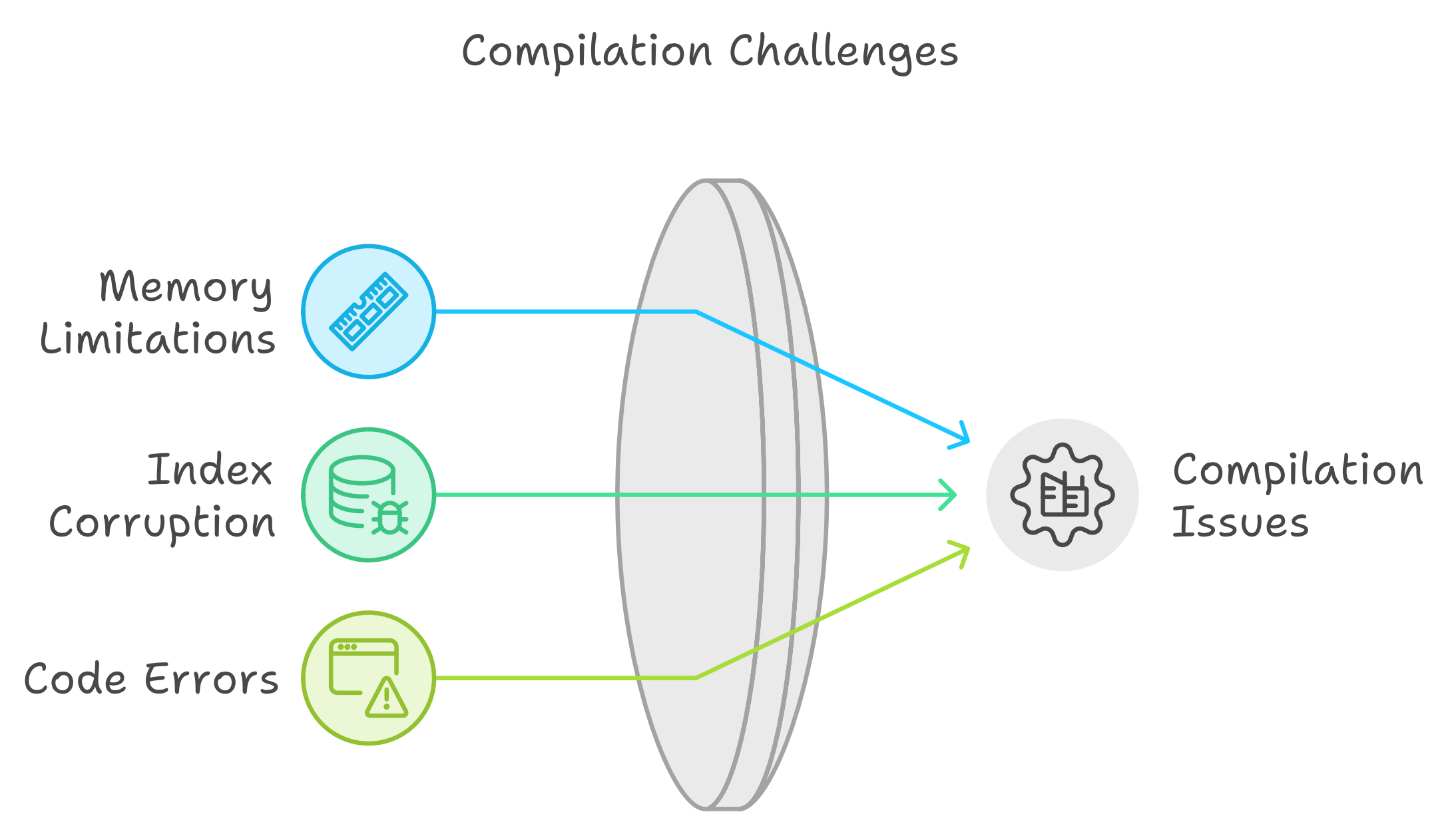
1. **Fix your Code!**
2. If you encounter dependency issues, try to pull from the develop branch and attempt the build again. Additionally, use **clean +** to remove old jars and retry.
3. Ensure that the **MAVEN_OPTS** variable is set to at least **1024m** to allocate sufficient memory.
4. Use **clean +** to terminate all Java processes, which should resolve any locking issues on jars.
5. On Micro-Services use “clms” to cleanup old jars

2. Building in IntelliJ

Only after a successful compilation in the command line should you attempt to build/run the project in IntelliJ.

2.1 Build in IntelliJ: Potential Problems

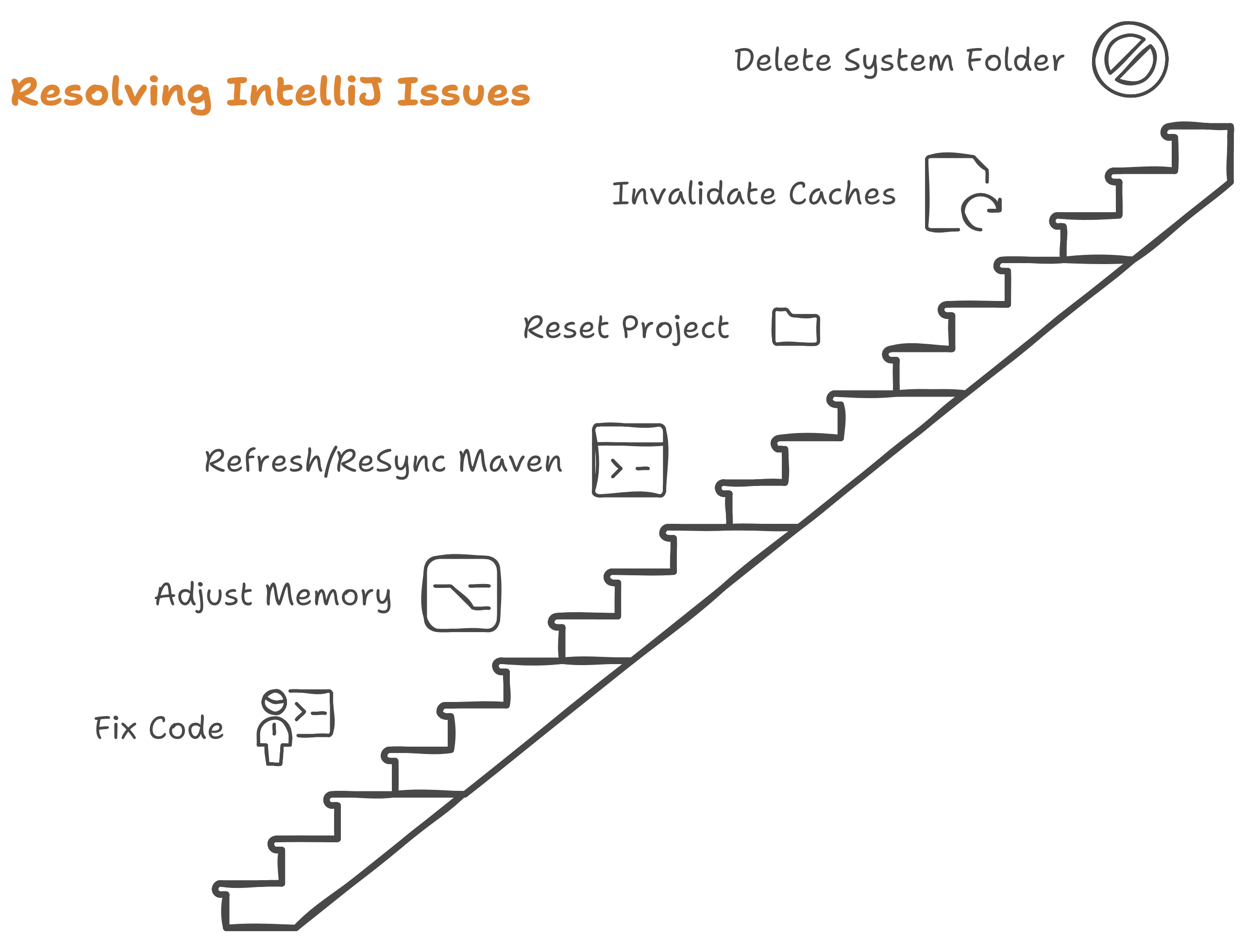
1. **Compilation errors in some classes**
2. **Compilation problems due to memory**
3. **Compilation problems due to index issues**

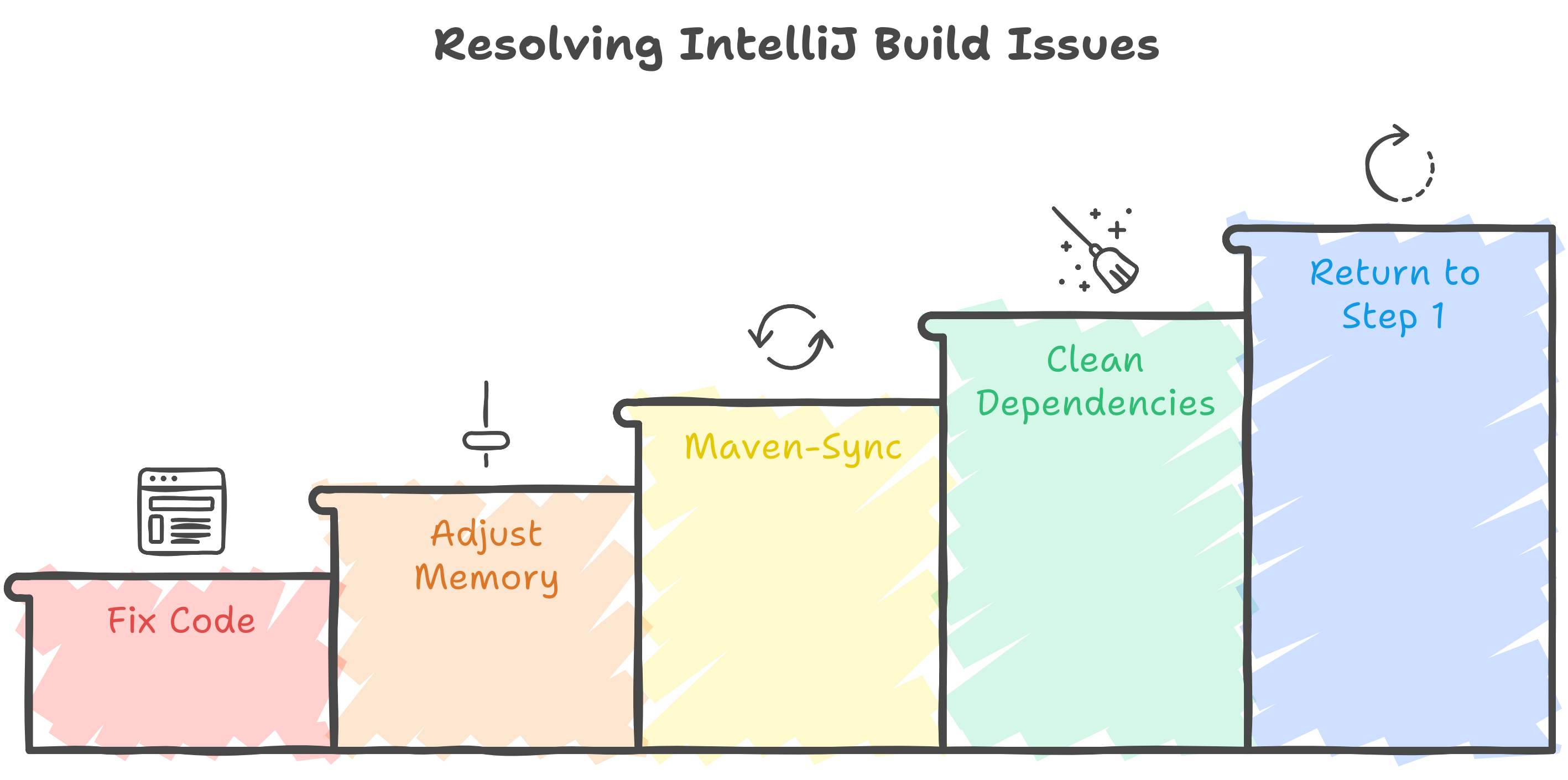


Solutions

1. **Fix your Code!**
2. Ensure that the memory allocated for the compiler is at least **1024m**. Check this in: **Settings -> Compiler -> Build Process -> Memory/Heap**.
3. Index problems can be challenging to resolve. Always start with Maven-Sync, which resolves most issues. If problems persist, run **clean +** in the command line to fully clean dependencies, then return to Step 1 (MVI/MVC).
4. Need to reset project , deletion of the .idea is necessary.Close the project, go to the project folder, rename the .idea (just add _ or something)Reopen the project.In Intellij 2024 - delete of the project also is needed, use right-click on the project name (when closed, in the window showing all recent project) and remove.IMPORTANT - shelves are saved in the .idea - do not delete the .idea before backup it up.

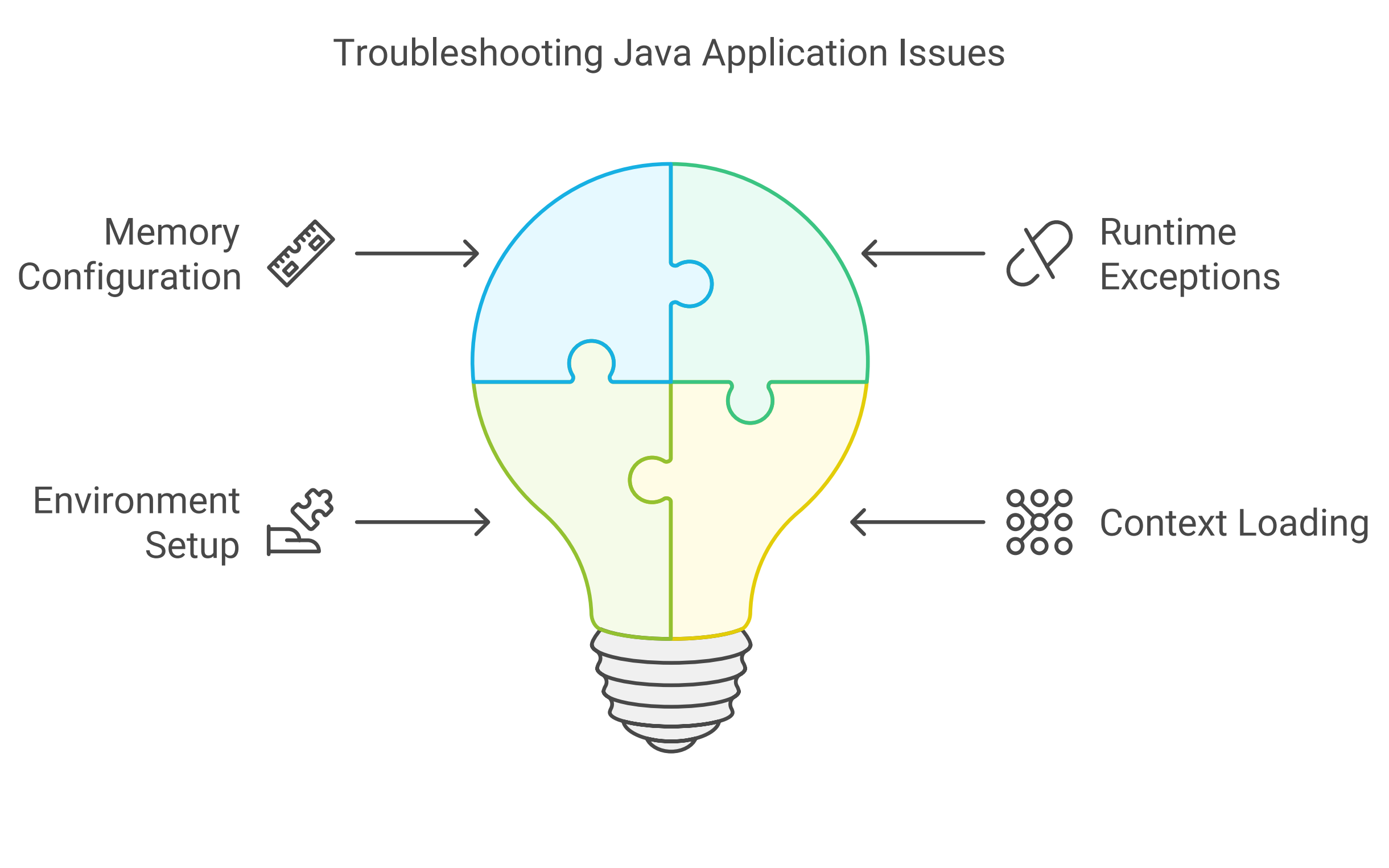
5. Invalidate caches - In file menu use the invalidate caches option. it may take time to build again all cache - fixes 99% of index problems if previous steps didn’t help.
6. Delete Intellij’s SYSTEM folder- under config / look for your intellij config folder and delete the System folder. Intellij must be closed for this operation. sometimes a restart for the computer is also needed to release some locking on certain files/folders under the System folder - fixes 100% of index problems but will take the most time to complete.





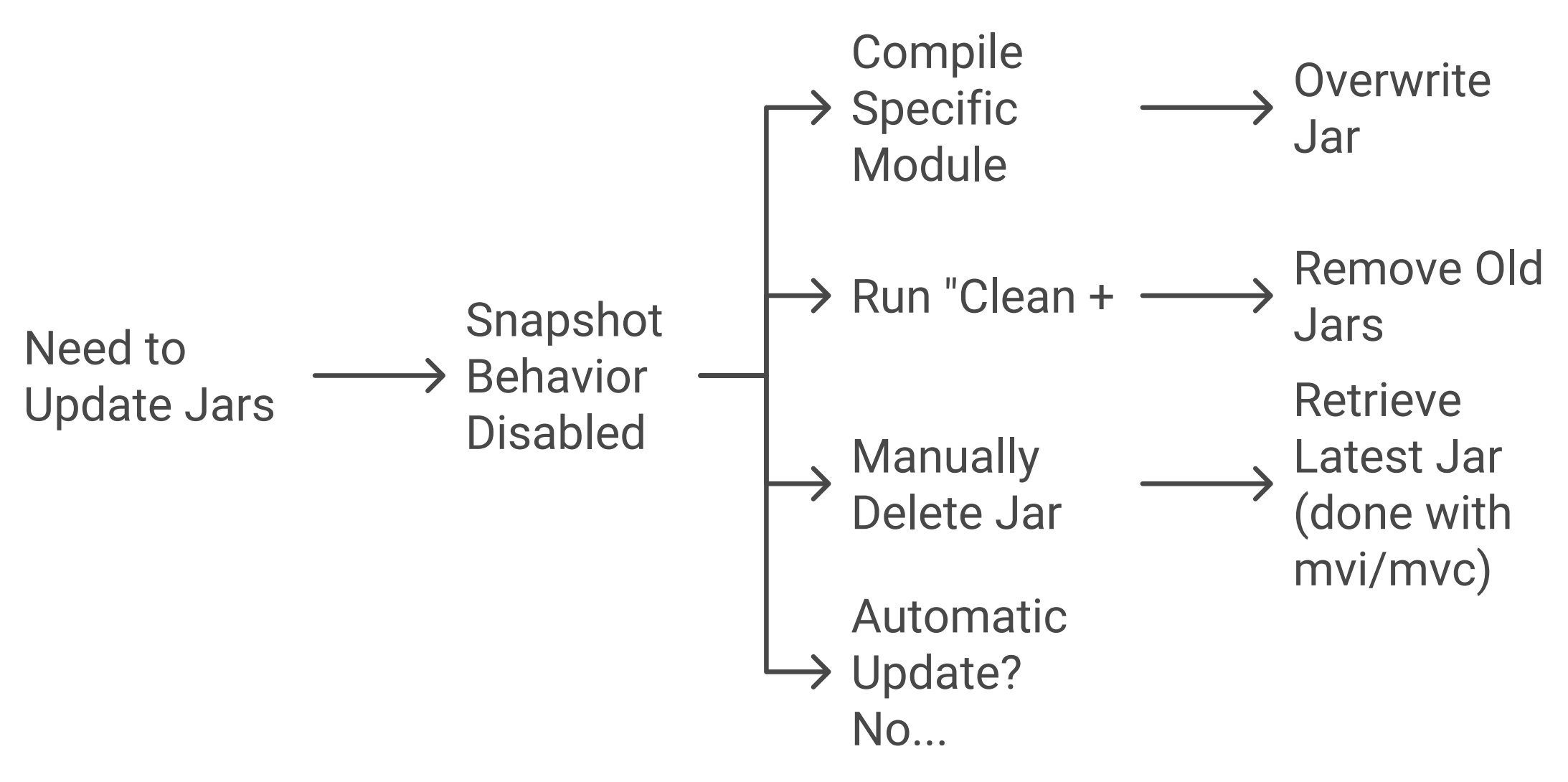
2.2 Running Problems

1. **Running fails with memory issues after the server starts**: The CRM server requires at least **1536m**, while JIDI is fine with**1024m**. Adjust the values in the running configuration for each server using **-Xmx**.
2. **Runtime exceptions with serialVersion issues**: This occurs when a call is made from CRM/ClientZone to JIDI via RMI, and changes are made on one side but not the other (service called with **@IDIService**).
3. **Environment not set or loaded correctly**: This can lead to various issues with data or behavior. Check the logs to verify which environment is loaded.
4. **Context not properly loaded**: Injection/autowired failures may occur if beans are not marked correctly. Use **@Inject** for Guice services and **@Autowired** for Spring beans. While **@Javax** can be used, it is not recommended as both Spring and Guice support this annotation. It is best practice to clarify the container from which the service/bean was injected.



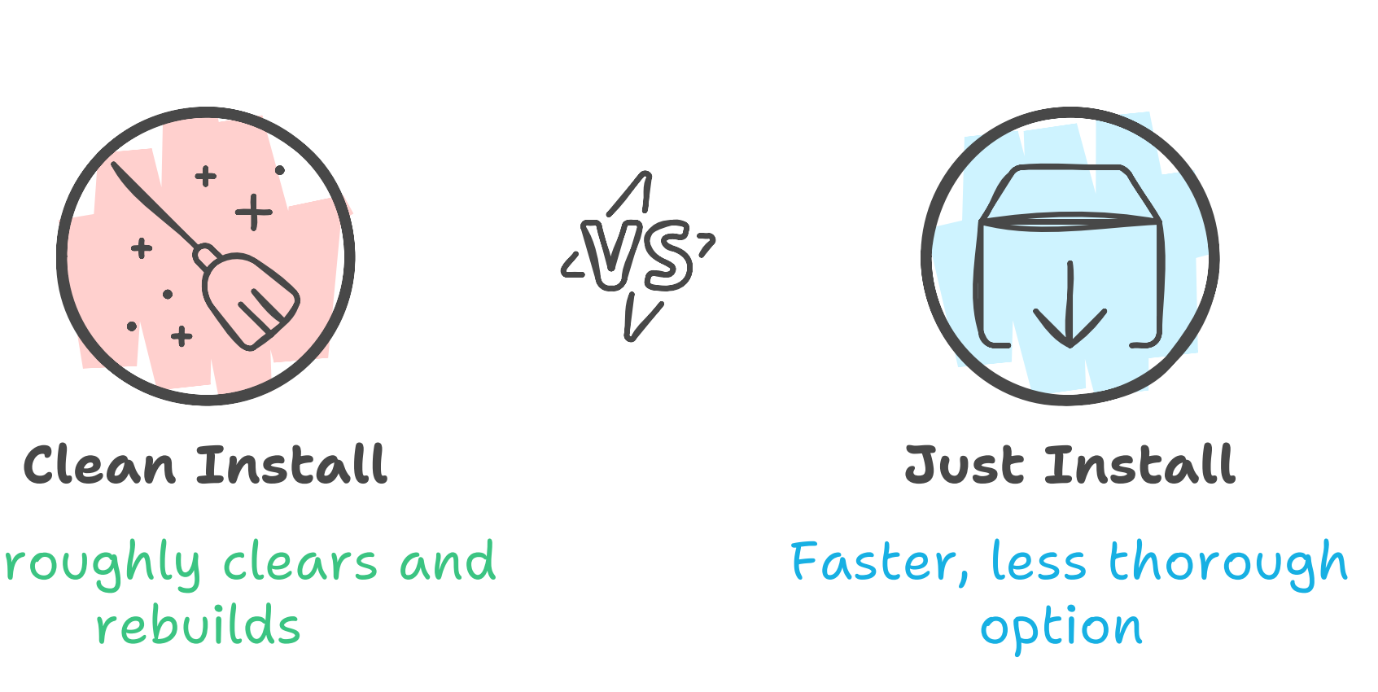
Need to know facts -

Mvi/mvc -U is not enough to update jars?Well no, not in our house. Snapshot behavior is disabled in artifactory in IDI, so jars once downloaded to the local .m3 repo will not be updated automatically. (which is maven’s default behavior)You can - 1. Compile the specific module to create the jar and over writing the jar.2. Run "Clean +" that cleans old jars (looks for jars older then 1 day)3. Delete manually the jar from the local repo, and get the latest jar from artifactory.

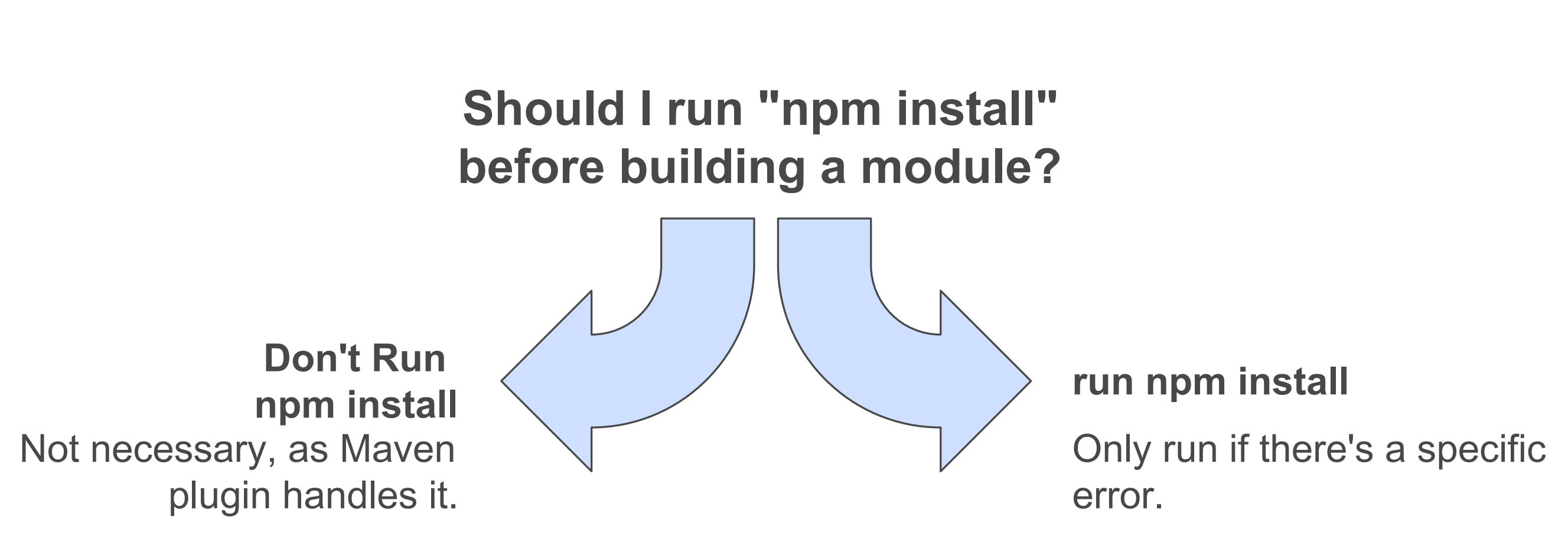


mvi or mvc? (clean install or just install)

mvn clean install will first clear the target folder in each module and then will run mvn install. Most of the time the cleaning of the target folders is redundant and mvn install will recompile all classes.
- Only if a class is moved manually (Not via intellij that does auto refactoring) then a duplicate class can remain
in source code & then in compiled form, and can cause runtime exception. mostly a compiled class left in
the target will not be loaded, hence won't cause problems. but will remain in the target and may be also in the jar, so clean should be done
- If a Query is moved (in *-queries.xml files) to another, the mvn clean is mandatory to remove the old* -queries.xml file, so
no duplicate will be detected only when starting the server. this is not handled by Intellij auto refactoring.
performance difference is not really high between install only or clean+install. so, could do either in that matter.



npm install first? or not at all?
- in each module under pearl that contains a package.json file, there's a maven-plugin that runs npm-install, so no npm install specifically, is necessary.
only if an error is caused that shows the maven-exec plugin FAILURE, one should run npm install so the error will be shown more clearly.


Should I run "npm install" before building a module?
Don't Run npm install
Not necessary, as Maven plugin handles it.
run npm install
Only run if there's a specific error.